

SQLD 스터디 2주차

JOIN · GROUP BY · HAVING · 서브쿼리



최진성

백엔드 개발자

Experience.

한국정보처리학회 주관 ACK 2025 논문 게재
A&I 3기 수료
스터디그룹 팀장

Bio.

“오늘보다 나은 코드를 고민하는 개발자, 최진성입니다.”

Contact.

Blog : <https://j-log.vercel.app/>

Github : <https://github.com/Theron02>

Profile.

인덕대학교 컴퓨터소프트웨어학과 (3.91/4.5)

이번 주 목표

1

지난 주 정답 확인

실습 문제 Q1~Q4 정답을 같이
확인합니다

2

JOIN

여러 테이블을 연결해서 한 번에
조회합니다

3

GROUP BY

집계 함수로 데이터를
그룹화합니다

4

HAVING & 서브쿼리

그룹 조건과 중첩 쿼리를
작성합니다

지난 주 실습 문제 정답

Q1. 나이가 21 살 초과인 학생을 모두 조회하시오 .

```
SELECT * FROM students  
WHERE age > 21;
```

**Q2. 전공이 컴퓨터소프트웨어인 학생을
이름 오름차순으로 정렬해서 조회하시오 .**

```
SELECT * FROM students  
WHERE major = ' 컴퓨터소프트웨어 '  
ORDER BY name ASC;
```

Q3. 이름에 ' 이 ' 가 들어가는 학생을 조회하시오 .

```
SELECT * FROM students  
WHERE name LIKE '% 이 %';
```

Q4. 나이순 정렬 후 가장 어린 학생 2 명만 조회하시오 .

```
SELECT * FROM students  
ORDER BY age ASC  
LIMIT 2;
```

실습 준비 — scores 테이블 생성

```
CREATE TABLE scores (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT,  
  subject VARCHAR(50),  
  score INT,  
  FOREIGN KEY (student_id)  
    REFERENCES students(id)  
);
```

```
INSERT INTO scores (student_id, subject, score) VALUES  
(1, 'SQL', 88),  
(1, '데이터모델링', 92),  
(2, 'SQL', 75),  
(2, '데이터모델링', 80),  
(3, 'SQL', 95),  
(3, '데이터모델링', 70),  
(4, 'SQL', 60),  
(4, '데이터모델링', 65),  
(5, 'SQL', 85),  
(5, '데이터모델링', 90),  
(6, 'SQL', 78),  
(6, '데이터모델링', 82);
```

FOREIGN KEY — student_id 가 students.id 를 참조합니다 . 존재하지 않는 student_id 를 넣으면 오류가 발생합니다 .

JOIN

두 개 이상의 테이블을 연결해서 한 번에 조회하는 문법입니다 .

종류	설명
INNER JOIN	두 테이블에서 조건이 일치하는 행만 조회합니다
LEFT JOIN	왼쪽 테이블은 전부 , 오른쪽은 일치하는 것만 조회합니다
RIGHT JOIN	오른쪽 테이블은 전부 , 왼쪽은 일치하는 것만 조회합니다

INNER JOIN 예시

```
SELECT s.name, sc.subject, sc.score
FROM students s
INNER JOIN scores sc ON s.id = sc.student_id;
```

LEFT JOIN 예시

```
SELECT s.name, sc.subject, sc.score
FROM students s
LEFT JOIN scores sc ON s.id = sc.student_id;
```

GROUP BY & 집계 함수

함수	설명
COUNT(*)	행 개수를 셉니다
SUM(컬럼)	합계를 구합니다
AVG(컬럼)	평균을 구합니다
MAX / MIN(컬럼)	최댓값 / 최솟값을 구합니다

전공별 학생 수

```
SELECT major, COUNT(*) AS 학생수
FROM students
GROUP BY major;
```

학생별 평균 점수

```
SELECT s.name, AVG(sc.score) AS 평균점수
FROM students s
INNER JOIN scores sc ON s.id = sc.student_id
GROUP BY s.id, s.name;
```

과목별 최고 / 최저 점수

```
SELECT subject,
       MAX(score) AS 최고점수 ,
       MIN(score) AS 최저점수
FROM scores GROUP BY subject;
```

HAVING

WHERE 는 GROUP BY 이전 , HAVING 은 GROUP BY 이후에 조건을 겁니다 .

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

평균 점수가 80 점 이상인 학생만 조회

```
SELECT s.name, AVG(sc.score) AS 평균점수
FROM students s
INNER JOIN scores sc ON s.id = sc.student_id
GROUP BY s.id, s.name
HAVING AVG(sc.score) >= 80;
```

학생 수가 1 명 초과인 전공만 조회

```
SELECT major, COUNT(*) AS 학생수
FROM students
GROUP BY major
HAVING COUNT(*) > 1;
```

 WHERE AVG(sc.score) >= 80 처럼 집계 함수에 WHERE 를 쓰면 오류가 발생합니다 . 집계 결과 조건은 반드시 HAVING 을 사용하세요 .

서브쿼리

쿼리 안에 또 다른 쿼리를 넣는 것입니다 . 괄호로 감싸서 사용합니다 .

WHERE 절 서브쿼리 — 전체 평균보다 높은 점수 조회

```
SELECT s.name, sc.score
FROM students s INNER JOIN scores sc ON s.id = sc.student_id
WHERE sc.score > (SELECT AVG(score) FROM scores);
```

FROM 절 서브쿼리 (인라인 뷰) — 평균 80 점 이상 학생 조회

```
SELECT name, 평균점수
FROM (
    SELECT s.name, AVG(sc.score) AS 평균점수
    FROM students s
    INNER JOIN scores sc ON s.id = sc.student_id
    GROUP BY s.id, s.name
) AS avg_table
WHERE 평균점수 >= 80;
```

FROM 절 서브쿼리는 반드시 별칭 (AS avg_table) 을 붙여야 합니다 .

이번 주 실습 문제

Q1 students 와 scores 테이블을 조인해서 학생 이름 , 과목 , 점수를 모두 조회하시오 .

Q2 학생별 총점을 구하고 총점 내림차순으로 정렬하시오 .

Q3 과목별 평균 점수를 구하시오 . (소수점 첫째 자리까지)

Q4 평균 점수가 80 점 미만인 학생의 이름과 평균 점수를 조회하시오 .

Q5 전체 평균 점수보다 높은 점수를 받은 학생의 이름과 점수를 조회하시오 . (서브쿼리 사용)

Q6 학생 수가 2 명 이상인 전공과 학생 수를 조회하시오 .

다음 주 예고

DDL

CREATE, ALTER, DROP
테이블 구조를 정의하고 수정합니다

DML

INSERT, UPDATE, DELETE
데이터를 조작합니다

트랜잭션

COMMIT, ROLLBACK
작업 단위를 관리합니다